

Topology e service slicing in SDN con qualità del servizio

Pierluigi Mase matricola M63001710

1 Introduzione

L'elaborato affronta due degli argomenti indicati nelle linee guida, *topology slicing* e *service slicing* in SDN, più il monitoraggio del traffico raccolto direttamente dal controller.

La rete è emulata con Mininet e poi controllata da un'applicazione Ryu, che si appoggia su OpenFlow 1.3. Su questa rete viene realizzato il topology slicing, che vincola due coppie di host ad usare un predeterminato percorso fisico per comunicare, impedendo di usarne altri. Il service slicing suddivide il traffico in tre classi, garantendo una banda minima a ciascuna di esse.

Per realizzare l'isolamento tra classi di traffico si utilizza la marcatura DSCP, risultato di una classificazione fatta su protocollo e porta di destinazione. L'isolamento non deriva dalla separazione dei percorsi, ma da garanzie di banda distinte per ciascuna classe, che restano valide anche quando le classi condividono lo stesso link. La marcatura viene fatta solamente sugli switch di bordo. Ogni switch, di bordo o di core, applica poi un per-hop behaviour: legge il DSCP e mette il pacchetto in una delle tre code di conseguenza. A quel punto lo scheduler HTB gestisce le code, servendole secondo le garanzie dichiarate. Le code sono create con `ovs-vsctl` e non dal controller, perché OpenFlow non possiede meccanismi di scheduling.

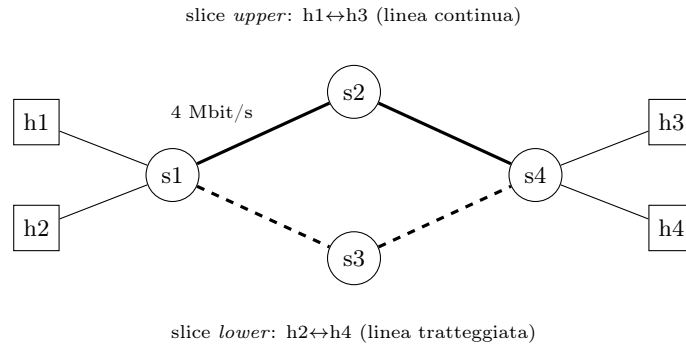


Figura 1: Topologia emulata. Il ciclo è deliberato: consente di assegnare percorsi disgiunti alle due slice, senza che un ramo venga disattivato da uno spanning tree.

2 Architettura

Pipeline OpenFlow. Ogni switch espone tre tabelle in cascata, ciascuna dedicata a una sola domanda.

Tabella	Match	Azione	regole bordo	regole core
0 — classificazione	protocollo, porta dest.	scrive il DSCP	4	1
1 — mappatura QoS	solo DSCP	seleziona la coda	3	3
2 — inoltra	coppia di MAC	porta di uscita	6	3

Le quattro regole della tabella 0 al bordo sono due di classificazione, una di azzeramento del DSCP e la regola di default; nel core resta solo quest'ultima, poiché gli switch interni non conoscono protocolli né porte applicative.

La separazione in tabelle riprende l'argomento di scalabilità di DiffServ: i nodi di bordo conoscono le porte applicative e i protocolli (tcp/udp), mentre i nodi core operano unicamente sul campo DSCP. Lo stato installato sui dispositivi di rete cresce con il numero di slice di servizio (tabella 1) e di slice di topologia (tabella 2) e non dipende dal numero di flussi attivi.

I dispositivi di bordo classificano su (protocollo, porta destinazione). Questo vuol dire che ho un numero di regole inferiore rispetto al modello DiffServ scalando come il core (numero di classi e non di flussi), ma non ho la possibilità di fare traffic conditioning. Ad esempio, non posso monitorare che una specifica sorgente rispetti il suo profilo contrattuale, né prendere azioni correttive di conseguenza. La marcatura DSCP viene eseguita con `APPLY_ACTIONS`, per cui la tabella successiva vedrà il campo già scritto.

Sugli switch di bordo la tabella 0 contiene inoltre una regola che azzerava il DSCP del traffico IP non classificato: la marcatura deve essere imposta dal bordo e non dalla sorgente, che altrimenti potrebbe assegnarsi una classe privilegiata in autonomia.

Tabella 2: Isolamento e assenza di flooding. Le regole di inoltro fanno match sulla coppia (sorgente, destinazione) a livello MAC: il traffico fra host di slice diverse non trova corrispondenza e viene scartato dalla regola di table-miss. Tutte le regole sono installate proattivamente alla connessione dello switch; il controller non apprende nulla e non usa mai `OFPP_FLOOD`. Le richieste ARP salgono al controller, che risponde impersonando l'host destinatario solo se i due appartengono alla stessa slice. L'assenza di flooding deriva da due fatti: le ARP non vengono propagate, ma sono gestite dal controller; inoltre le regole sono proattive, quindi non ci sarà mai una destinazione sconosciuta. Questo rende innocuo il ciclo in topologia, senza bisogno di spanning tree.

Classi di servizio e admission control. Le code sono create su Open vSwitch tramite `ovs-vsctl`, quindi fuori dal protocollo OpenFlow, su entrambe le direzioni di ogni link inter-switch.

Classe	Match	DSCP	Coda	Garanzia	Tetto
realtime	UDP porta 5001	46 (EF)	2	0,8 Mbit/s	1,2 Mbit/s
business	UDP porta 5002	18 (AF)	1	2,0 Mbit/s	—
best effort	tutto il resto	0	0	0,4 Mbit/s	—

La somma delle garanzie, 3,2 Mbit/s, non eccede la capacità del link: la verifica è svolta all'avvio e rifiuta configurazioni irrealizzabili. Si tratta di un controllo di ammissione *sulle classi*, statico, non della *call admission control* per flusso fatta da IntServ, che richiederebbe il protocollo di segnalazione a runtime e stato per flusso in ogni nodo. Ne consegue che la garanzia è deterministica sull'aggregato di classe, ma si ripartisce fra i flussi che la condividono. Negli esperimenti condotti le due cose coincidono, poiché ogni classe è alimentata da un solo flusso. Il margine residuo non è sprecato, perché HTB è work-conserving e presta alle altre classi la capacità non utilizzata.

Perché non il limitatore di banda di Mininet. Negli esempi d'uso di Mininet la capacità di un link si impone con il parametro `bw` di `TCLink`, che installa una gerarchia HTB con una sola classe. Qui non è sufficiente, ci servono tre code distinte, selezionabili con l'azione `set_queue`. Il legame tra numero di coda OpenFlow e classe HTB è mantenuto nel database di Open vSwitch, quindi le code vanno create attraverso la sua interfaccia QoS. Poiché su una stessa interfaccia può esistere un solo qdisc radice, e il secondo a essere installato sovrascrive il primo senza segnalarlo, si è scelto che un solo componente lo controlli per ciascuna interfaccia: Mininet sui link di accesso, dove impone il ritardo di base, e Open vSwitch sui link interni, dove crea le code.

Configurazione dichiarativa. Classi, code, percorsi, mappa delle porte, host e parametri sperimentali sono raccolti in un unico file YAML, letto sia dalla topologia sia dal controller. Modificare la configurazione non richiede di toccare il codice.

3 Esperimenti

Si confrontano due scenari, a parità di topologia, capacità e traffico generato. Nello scenario *baseline* il controller non marca e non assegna code: tutto il traffico condivide una singola coda FIFO. Nello scenario *slicing + QoS* il traffico è suddiviso per classe. Cambia una sola variabile (assegnazione di DSCP presente o assente), così che ogni differenza osservata sia attribuibile alla disciplina di accodamento.

Il traffico è generato con D-ITG da h1 verso h3, con tre flussi UDP simultanei da 0,7, 3 e 3 Mbit/s su pacchetti da 1400 byte, per 60 secondi. Il carico offerto, 6,7 Mbit/s, eccede la capacità del link: in assenza di congestione lo scheduler non avrebbe nulla da arbitrare, e i due scenari risulterebbero indistinguibili. La scelta di UDP anziché TCP elimina il fenomeno del controllo di congestione, rendendo il carico offerto pari a quello dichiarato.

Verifica del topology slicing. Il comando `pingall` riporta 66% dropped (4/12 received): ogni host raggiunge esclusivamente il proprio pari di slice. Il fallimento avviene già in fase di risoluzione degli indirizzi, poiché il controller non risponde alle richieste ARP fra slice diverse. I contatori per regola (`dump-flows`) e la cattura del traffico confermano che h1→h3 attraversa s2 e h2→h4 attraversa s3.

h1 ping h3

```
$ ovs-ofctl -O OpenFlow13 dump-flows s2 table=2
table=2, n_packets=5, priority=10,dl_src=...:01,dl_dst=...:03 actions=output:"s2-eth2"
table=2, n_packets=5, priority=10,dl_src=...:03,dl_dst=...:01 actions=output:"s2-eth1"
```

```

table=2, n_packets=0, priority=0 actions=drop

$ ovs-ofctl -O OpenFlow13 dump-flows s3 table=2
table=2, n_packets=0, priority=10,dl_src=...:02,dl_dst=...:04 actions=output:"s3-eth2"
table=2, n_packets=0, priority=10,dl_src=...:04,dl_dst=...:02 actions=output:"s3-eth1"
table=2, n_packets=0, priority=0 actions=drop

```

h2 ping h4

```

$ ovs-ofctl -O OpenFlow13 dump-flows s2 table=2
table=2, n_packets=0, priority=10,dl_src=...:01,dl_dst=...:03 actions=output:"s2-eth2"
table=2, n_packets=0, priority=10,dl_src=...:03,dl_dst=...:01 actions=output:"s2-eth1"
table=2, n_packets=0, priority=0 actions=drop

$ ovs-ofctl -O OpenFlow13 dump-flows s3 table=2
table=2, n_packets=5, priority=10,dl_src=...:02,dl_dst=...:04 actions=output:"s3-eth2"
table=2, n_packets=5, priority=10,dl_src=...:04,dl_dst=...:02 actions=output:"s3-eth1"
table=2, n_packets=0, priority=0 actions=drop

```

Un terzo esperimento carica la slice inferiore con un flusso h2→h4 servito senza perdite sul proprio percorso. Le classi garantite della slice superiore conservano la banda e l'assenza di scarti, con ritardi che restano nell'ordine dei millisecondi: i due tenant risultano isolati anche sotto carico. I due percorsi non condividono alcun link, e sugli switch di bordo comuni impiegano porte di uscita distinte, quindi code separate.

4 Risultati

Classe	Baseline			Slicing + QoS		
	banda	ritardo	perdita	banda	ritardo	perdita
realtime	417 Kbit/s	2915 ms	28,5 %	600 Kbit/s	5,1 ms	0 %
business	1506 Kbit/s	2933 ms	28,2 %	2120 Kbit/s	14,1 ms	0 %
best effort	1530 Kbit/s	2932 ms	27,3 %	887 Kbit/s	11431 ms	56,0 %
totale	3453 Kbit/s			3607 Kbit/s		

Nel baseline le tre classi sono indistinguibili: ritardo entro l'1 % di scostamento e perdita compresa fra il 27,3 e il 28,5 % per tutte. È il comportamento atteso da una coda unica, che non conosce differenze fra i pacchetti che accoda.

Con lo slicing attivo la classe realtime passa da 2915 a 5,1 ms, un fattore superiore a 500, e da oltre il 28 % di perdita a zero. La classe business ottiene la propria banda garantita, anch'essa senza perdite. Il degrado non scompare: si concentra interamente sulla classe priva di garanzie.

Osservando il throughput aggregato, comparabile nei due scenari (3,45 contro 3,61 Mbit/s) è evidente che lo scheduler non aumenta la capacità disponibile e non ne spreca: **redistribuisce il degrado** verso la classe che, per contratto, è la più sfavorita. La qualità del servizio è un meccanismo di allocazione e non di espansione delle risorse.

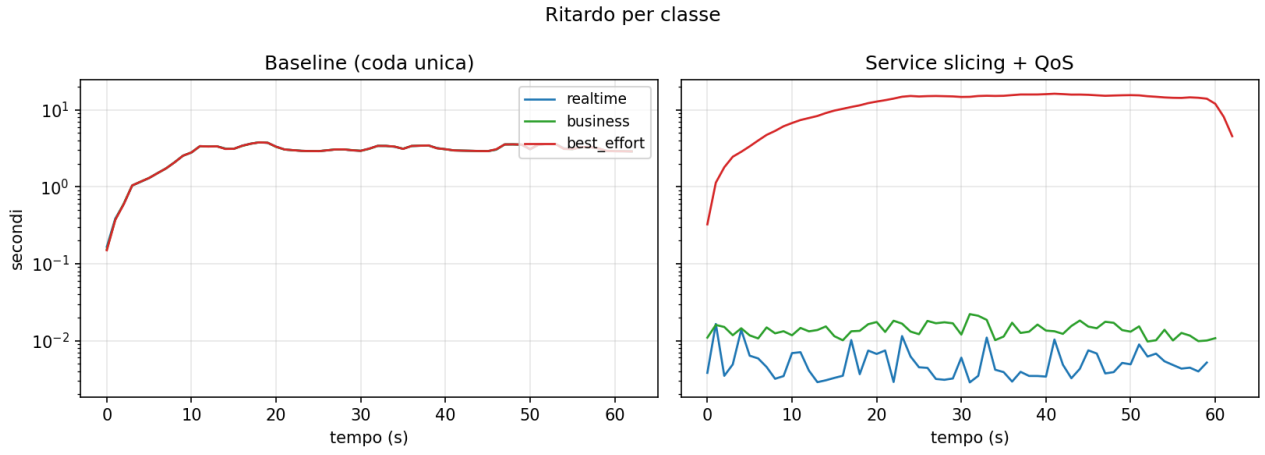


Figura 2: Ritardo per classe nel tempo, scala logaritmica. I tre flussi sono generati e registrati separatamente alla sorgente e al ricevitore; è la rete a non distinguerli. Nel pannello di sinistra le curve sono sovrapposte, perché la coda condivisa impone a tutti lo stesso trattamento; a destra si separano di tre ordini di grandezza.

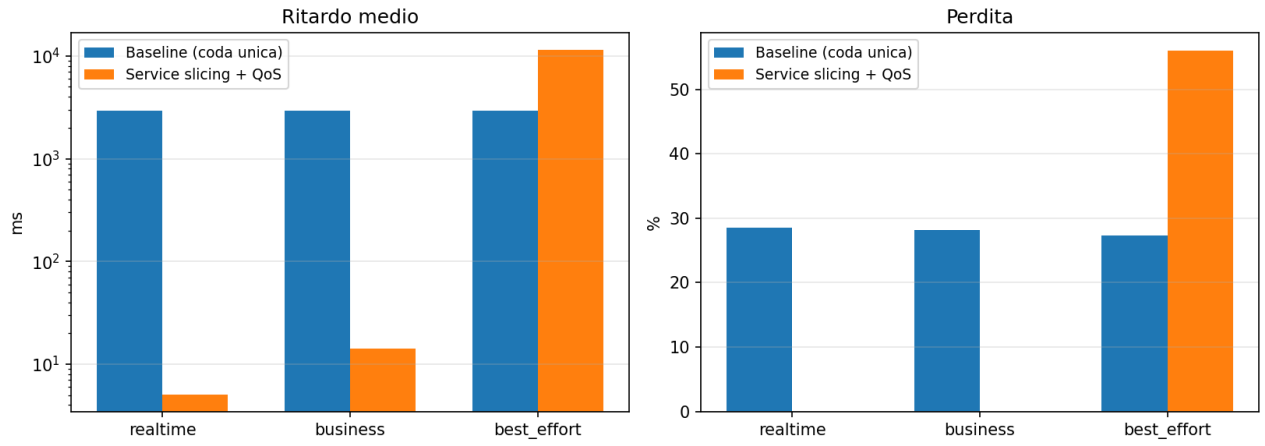


Figura 3: Ritardo medio (scala logaritmica) e perdita per classe. Nel baseline i tre valori coincidono entro l'1% per il ritardo. L'assenza di barra arancione per realtime e business indica perdita nulla.

Isolamento fra tenant. Il confronto fra lo scenario di riferimento e quello con la slice inferiore sotto carico:

Classe	slice inferiore scarica		slice inferiore carica	
	banda	perdita	banda	perdita
realtime	600 Kbit/s	0 %	625 Kbit/s	0 %
business	2120 Kbit/s	0 %	2318 Kbit/s	0 %

Il flusso h2→h4 ottiene 3027 Kbit/s senza perdite sul proprio percorso, mentre le classi garantite della slice superiore conservano banda e assenza di perdite.

5 Monitoraggio e comportamento dello scheduler

Il controller interroga periodicamente le statistiche per coda (`OFPPQueueStatsRequest`) e converte i contatori cumulativi in throughput. La figura 4 mostra il fenomeno dal punto di vista dello switch, anziché da quello del ricevitore: nel baseline una sola coda trasporta l'intero carico, mentre nello scenario completo la ripartizione segue le garanzie dichiarate. La risalita finale della classe best effort corrisponde allo svuotamento della coda dopo il termine degli altri flussi.

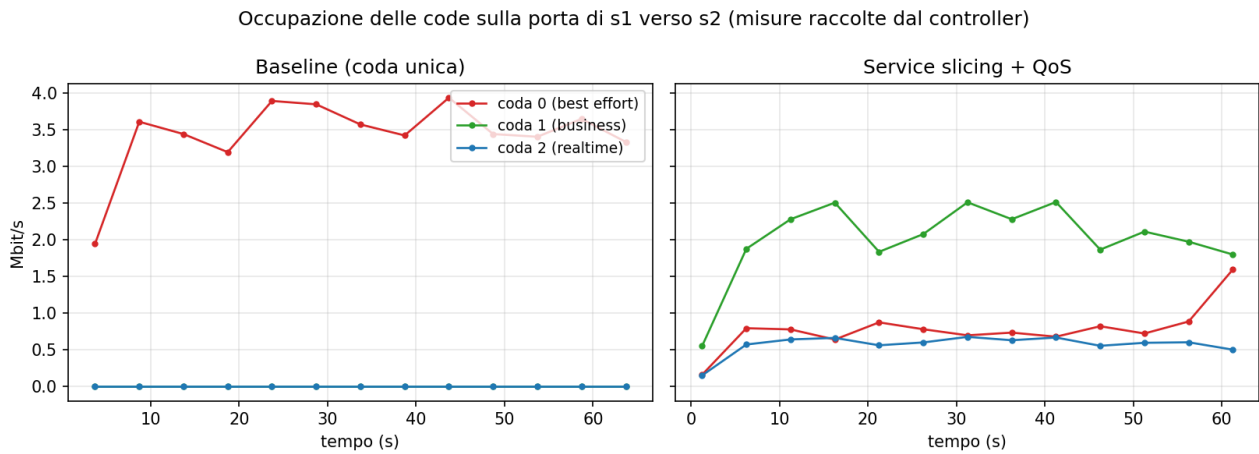


Figura 4: Occupazione delle code sulla porta congestionata, misurata dal modulo di monitoraggio del controller.

I contatori di HTB documentano il comportamento work-conserving (indicato come dynamic slicing) richiesto dalla traccia come estensione. Nello scenario completo la classe best effort è servita con 2170 pacchetti entro la propria quota e 2849 presi in prestito dalle classi che non stanno usando la banda riservata; la classe realtime, al contrario, non ricorre mai al prestito e non subisce scarti. Quindi il traffico non prioritario utilizza la capacità libera, ma la cede appena le classi garantite la reclamano.

6 Esecuzione e verifica

Ambiente. Ubuntu 20.04 (immagine VM Mininet 2.3.0), Open vSwitch 2.13, Python 3.8, Ryu 4.34, D-ITG 2.8.1, matplotlib per le figure.

Ryu 4.34 non fissa la versione di `eventlet` e non si avvia su Python 3.10 o superiore. Le dipendenze si installano con:

```
pip3 install ryu 'eventlet==0.30.2' pyyaml matplotlib
```

D-ITG non è nei repository Ubuntu 20.04 e va compilato dai sorgenti trovati in rete.

Struttura del repository.

<code>policy/slices.yaml</code>	configurazione dichiarativa dell'intero sistema
<code>topologia/topo_slicing.py</code>	topologia Mininet e creazione delle code
<code>controller/slicing_controller.py</code>	applicazione Ryu
<code>esperimenti/esegui.py</code>	esecuzione di uno scenario
<code>esperimenti/grafici.py</code>	generazione delle figure
<code>risultati/</code>	log, misure e grafici

Riproduzione degli esperimenti. I comandi vanno lanciati dalla radice del repository. Servono due terminali: nel primo resta in ascolto il controller, nel secondo si avviano topologia e traffico. Ogni scenario dura circa 70 secondi.

```
# scenario con slicing e QoS
ryu-manager controller/slicing_controller.py
sudo python3 esperimenti/esegui.py --nome slicing_qos_finale

# scenario baseline
SLICING_BASELINE=1 ryu-manager controller/slicing_controller.py
sudo python3 esperimenti/esegui.py --nome slicing_baseline_finale

# isolamento fra tenant
ryu-manager controller/slicing_controller.py
sudo python3 esperimenti/esegui.py --nome slicing_isolamento --incrociato

# figure
python3 esperimenti/grafici.py
```

Verifica del funzionamento. Avviando la topologia interattiva (`sudo python3 topologia/topo_slicing.py`) con il controller in ascolto:

Comando (dal prompt mininet>)	Esito atteso
<code>pingall</code>	66% dropped (4/12 received)
<code>sh ovs-ofctl -O OpenFlow13 dump-flows s1 table=0</code>	4 regole: 2 di classificazione, azzeramento del DSCP, table-miss
<code>sh ovs-ofctl -O OpenFlow13 dump-flows s2 table=0</code>	1 sola regola: il core non classifica
<code>sh ovs-ofctl -O OpenFlow13 dump-flows s2 table=2</code>	contatori non nulli sulle sole regole della slice <i>upper</i>
<code>sh tc -s class show dev s1-eth3</code>	tre classi con rate e ceil della policy

Materiale prodotto. Ogni cartella sotto `risultati/` contiene i log D-ITG per flusso, i riepiloghi testuali, le serie temporali a un secondo, i contatori HTB finali (`tc_class.txt`), una copia delle statistiche per coda raccolte dal controller (`queue_stats.csv`) e i metadati dell'esecuzione. Le tabelle e le figure di questo documento sono ricavate da `slicing_qos_finale` e `slicing_baseline_finale`.

7 Limiti e sviluppi

Aderenza parziale al modello DiffServ. L'architettura segue il modello diffserv (classificazione al bordo, per-hop behavior nel core) ma non implementa le sottoclassi AF con preferenza di scarto, né una precedenza assoluta per la classe EF: l'interfaccia OVSDb espone solo **min-rate** e **max-rate**, quindi la classe prioritaria dispone di banda garantita ma non riceve priorità di servizio.

Assenza di condizionamento del traffico (tabella 0 al bordo). Il token bucket è impiegato come meccanismo di scheduling, ma non come policer: non c'è un meter che verifichi che una sorgente rispetti il profilo della propria classe. Un esperimento dedicato mostra la conseguenza (`slicing_ef_saturato`): una sorgente che eccede il contratto degrada la propria classe. Per meglio aderire al modello DiffServ, andrebbe fatta una classificazione per flusso ai soli nodi di edge, il che permetterebbe l'aggiunta di policing per flusso (tramite meter OpenFlow).

Granularità delle regole di inoltri (tabella 2). Il match su coppia di indirizzi garantisce isolamento esplicito al costo di un numero di regole quadratico negli endpoint di ciascuna slice di topologia. Con slice fatte di più host si adotterebbe un'etichetta di slice inserita al bordo che identifica un percorso virtuale nella slice, così che il core inoltri in funzione di essa, usando una sola regola per percorso nella slice di topologia invece che per coppia di endpoint.

Classificazione unidirezionale. Il match avviene sulla porta di destinazione, che nel verso di ritorno diventa porta sorgente: le risposte non trovano corrispondenza e ricadono nella classe di default. Negli esperimenti il fenomeno non emerge, poiché il traffico UDP generato è unidirezionale, ma con TCP gli ACK viaggierebbero in best effort e il loro ritardo rallenterebbe il mittente nonostante la banda garantita nel verso dei dati. Una classificazione bidirezionale richiederebbe, per ciascuna classe, una seconda regola con porte sorgente e destinazione invertite.